

Low-Latency Go Cheatsheet

Lessons: [46 — Measuring & Allocation](#) · [47 — GC, Memory Layout & Contention](#) · [48 — Lock-Free, Zero-Copy & Tail Latency](#)

Examples: [46](#) · [47](#) · [48](#)

Covers: benchmarking, escape analysis, GC tuning, memory layout, contention, pprof, atomics, tail latency

Legend: [*] = API or flag the lessons have not covered yet

THE RULE: MEASURE, DON'T GUESS

latency	how long ONE operation takes
throughput	how many per second – they trade off
the mean lies	p50 says nothing about the user who waited 2s
p99 / p99.9	the tail is what people actually complain about
tail amplification	a request fanning out to 10 services hits the p99 of one almost every time
profile first	the bottleneck is never where you think
optimize the algorithm first	no micro-optimization beats removing an $O(n^2)$

BENCHMARKING

```
func BenchmarkX(b *testing.B) { for b.Loop() { ... } }    Go 1.24+, DCE-proof
for i := 0; i < b.N; i++      the classic form
b.ReportAllocs()              B/op and allocs/op – the numbers that matter
b.ResetTimer()                after setup
b.StopTimer() / b.StartTimer()
b.SetBytes(n)                 report MB/s
b.RunParallel(...)            measure under contention
go test -bench=. -benchmem -count=10 > new.txt
benchstat old.txt new.txt [*] the ONLY honest way to compare two runs
testing.AllocsPerRun(100, f)  allocations of one function
var sink T                    assign the result to a package var, or it's eliminated
(a benchmark that got optimized away reports 0.3 ns/op – always sanity-check)
```

ESCAPE ANALYSIS & ALLOCATION

go build -gcflags="-m" ./...	what escapes, and why
go build -gcflags="-m -m"	[*] the full reasoning chain
"escapes to heap"	one allocation per call
"moved to heap"	a local whose address outlived the frame
stack allocation is free	no GC involvement at all
what forces the heap	returning a pointer to a local (usually)
	storing into an interface (boxing)
	a closure capturing by reference
	a slice/map that grows beyond a known bound
	anything the compiler can't prove is bounded
preallocate	make([]T, 0, n) – one allocation instead of log(n)
strings.Builder	instead of += in a loop ($O(n^2)$ → $O(n)$)
	b.Grow(n) when you know the size
[]byte vs string	converting either way COPIES; work in one type
strconv.AppendInt(buf, ...)	format without allocating
sync.Pool	reuse buffers; RESET before Put, and expect losses at GC
generics vs any	a type parameter avoids the boxing an interface forces
map[string(b)]	THE elision: the compiler skips the string copy for
	m[string(byteSlice)] – a lookup with no allocation
	(only for a direct lookup; assigning it allocates)
runtime.ReadMemStats(&m)	Alloc / TotalAlloc / Mallocs / NumGC / PauseTotalNs
	– watch allocations drive the collector in real time
	(it stops the world; sample it, don't poll it hot)

THE GC

concurrent mark & sweep	it runs alongside your program; pauses are sub-ms
the cost is PROPORTIONAL to pointers	scanning is the work, not the byte count
GOGC=100	default: collect when the heap has doubled
GOGC=400	less frequent GC, more memory – a real throughput knob
GOGC=off	[*] with GOMEMLIMIT, for latency-critical batch work
GOMEMLIMIT=6GiB	a SOFT limit; the GC works harder as you approach it
	– the single best knob for a container with a memory cap
runtime/debug.SetGCPercent / SetMemoryLimit	the programmatic versions
fewer pointers = less GC	[]Item beats []*Item; an index beats a pointer
	a map[string]struct{...} with no pointers is never scanned
GODEBUG=gctrace=1	[*] one line per GC cycle, to stderr
allocation IS the cost	the fastest GC is the one with nothing to collect

MEMORY LAYOUT

struct field alignment	fields are padded to their alignment
struct{ a bool; b int64; c bool }	→ 24 bytes
struct{ b int64; a, c bool }	→ 16 bytes
order fields big → small	free memory, better cache density
unsafe.Sizeof / Alignof / Offsetof	[*] check it yourself
fieldalignment linter	[*] finds them automatically
AoS vs SoA	[]Point{X,Y} vs struct{ X, Y []float64 }
	SoA wins when you touch ONE field across many items
cache line = 64 bytes	the unit the CPU actually moves
locality beats cleverness	a linear scan of a slice beats pointer-chasing a tree
	at small n, every time

CONTENTION

sync.Mutex	fine until it isn't; measure before sharding
sync.RWMutex	helps only with MANY readers and long read sections – its own bookkeeping is more expensive than Mutex
sharding	N mutex+map pairs, keyed by hash(key)%N – the standard fix for a hot map
atomics	for counters and pointer swaps; no blocking at all
sync.Map	write-once/read-many keys ONLY; otherwise slower
false sharing	two hot variables in one 64-byte cache line ping-pong between cores – pad them apart
type padded struct { v atomic.Int64; _ [56]byte }	
per-goroutine state	the real fix: don't share, then merge at the end
channels are not free	a mutex is often cheaper than a channel for pure state

pprof & TRACING

```
import _ "net/http/pprof"      /debug/pprof/* on your admin port (never public)
go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30    CPU
go tool pprof http://localhost:6060/debug/pprof/heap                  heap
    inuse_space / alloc_space    what's live now / everything ever allocated
/debug/pprof/goroutine        every stack – the leak finder
/debug/pprof/mutex            contention (runtime.SetMutexProfileFraction)
/debug/pprof/block            blocking ops (runtime.SetBlockProfileRate)
(pprof) top / list Func / web  the three commands you need
-diff_base=old.pprof          [*] compare two profiles
go tool trace trace.out        the scheduler timeline: GC, blocking, goroutine states
runtime/trace Start/Stop      to capture it
```

LOCK-FREE & ZERO-COPY

atomic.Pointer[T]	typed pointer swap, no unsafe
copy-on-write	read a snapshot with Load(); to write: clone, modify, CompareAndSwap – readers never block, never lock
for { old := p.Load(); nw := clone(old); mutate(nw) if p.CompareAndSwap(old, nw) { break } }	
ideal for config, routing tables, feature flags – read constantly, written rarely	
atomic.Value	the pre-generics version
unsafe.String / unsafe.Slice	[*] []byte <=> string with NO copy – the buffer must never be mutated afterwards
io.Copy / io.CopyBuffer	streams without buffering the whole payload
bufio.Reader/Writer	amortize syscalls
net.Buffers	[*] writev: several buffers, one syscall
batching	one flush per N items instead of N flushes
singleflight	collapse duplicate concurrent work
ring buffer	fixed memory, no allocation per item

TAIL LATENCY ENGINEERING

GC pauses	sub-millisecond now; assist time is the real cost
hedged requests	send a second request at p95; take the first answer – a few % extra load buys a much better p99
deadlines everywhere	a request that can't finish in time should die early
load shedding	reject at the door instead of queueing
bounded queues	an unbounded queue converts overload into latency
GOMAXPROCS in containers	match it to the CPU limit, or the scheduler thrashes (automaxprocs, or set it explicitly)
warm the pools	first-request latency includes connection setup
QUIC / HTTP-3	[*] no head-of-line blocking across streams; 0-RTT resumption

TRAPS & MEMORIZE

optimizing without a profile	time spent on the 2%
benchmark eliminated by DCE	0.3 ns/op means it never ran
comparing single runs	use <code>-count=10</code> and <code>benchstat</code>
micro-optimizing before the algorithm	$O(n^2)$ with fast constants is still $O(n^2)$
RWMutex by reflex	slower than Mutex under short critical sections
sync.Map by reflex	slower than <code>map+RWMutex</code> for a normal write mix
sync.Pool without a reset	you hand out dirty buffers
pointer-heavy data	every pointer is GC scan work forever
<code>[]*T</code> where <code>[]T</code> would do	<code>n</code> allocations and <code>n</code> scan roots
GOMAXPROCS unset in a container	Go sees the host's cores, not the cgroup limit
no GOMEMLIMIT with a cgroup cap	the OOM killer finds you before the GC does
unsafe.String on a reused buffer	memory corruption, discovered in production
premature sharding	complexity for contention you never measured